SC4003 CE/CZ4046 INTELLIGENT AGENTS

ASSIGNMENT 1

SEAN NEO (U2321281G)

1.1. Method of Value Iteration

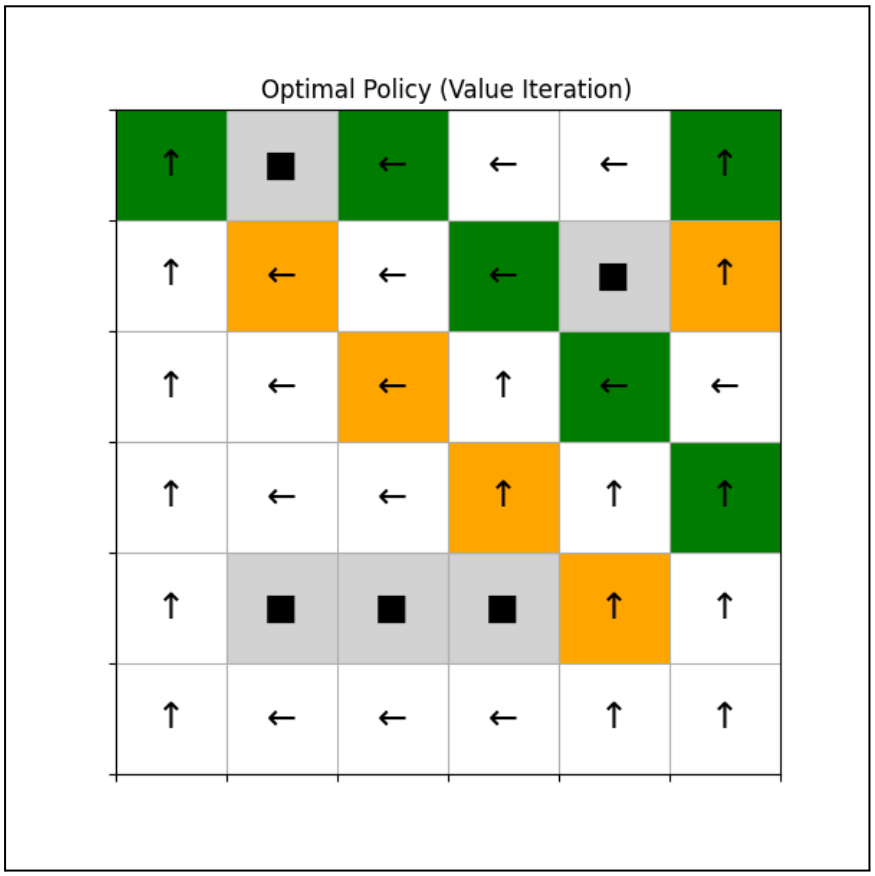
1.1.a. Description of implemented solution

The environment is a 6x6 grid with walls (None), green square (+1 reward), orange squares (-1 reward), and white squares (-0.05 reward). The transition model is stochastic: with probability 0.8 the agent moves in the intended direction, with probability 0.1 it moves in one of the two perpendicular directions.

The value iteration algorithm was implemented as follows:

- Initialization: The utility of every non-wall state is set to 0
- Bellman update: For each state s and gamma = 0.99,
$$U(s) = R(s) + \gamma \max_{a=\{U,D,L,R\}} \sum_{s'} P(s'|s,a) \cdot U(s')$$
- The expected utility for a given action a is computed by considering the intended move and the two perpendicular side movements, using the provided probabilities of 0.8 and 0.2. If a move is invalid (wall), the agent stays in place.
- Convergence: The update is repeated until the maximum change in utility across all states falls below 10^{-4}. In practice, this required about 150-200 iterations.
- Tracking: For the convergence plot, the utilities of five selected states were recorded after every iteration.

1.1.b. Plot of optimal policy



This table indicates the best action for the agent in every state in order to maximize long-term rewards. The arrows represent the direction the agent should move from each state.

1.1.c. Utilities of all states from Value Iteration

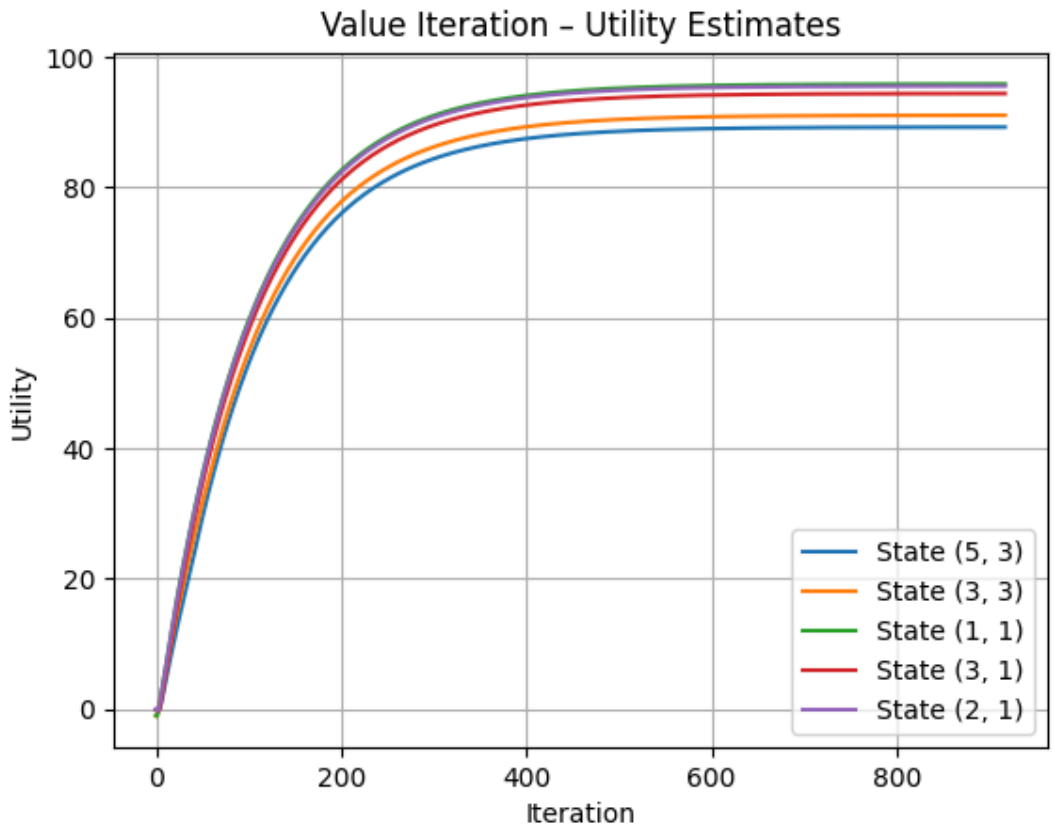
99.99	0	95.01	93.828	92.595	93.274
98.371	95.858	94.507	94.357	0	90.864
96.912	95.539	93.246	93.124	93.049	91.731
95.504	94.392	93.163	91.061	91.751	91.825
94.251	0	0	0	89.484	90.493
92.862	91.642	90.437	89.247	88.489	89.213

Given the discount factor of 0.99, the maximum possible discounted return if the agent received +1 every step is

$$\sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma} = \frac{1}{0.01} = 100$$

Values near 100 (e.g. 99.99 for the top-left +1 cell) indicates that the agent can stay in or quickly return to high-reward states. Slightly lower values reflect states that are farther from +1 cells or have a risk of slipping into -1 cells.

1.1.d. Plot of utility estimates as a function of the number of iterations



The figure shows how the estimated utilities of the five tracked states evolve during value iteration. Initially, all utilities are zero. As iterations progress, the values increase and eventually level off as they approach their optimal values. The shape of the curves is similar to Figure 17.5(a) in the textbook, which was also a rapid rise in the first few iterations followed by a gradual convergence.

I also noticed that states with a higher long-term reward potential (e.g. state (5,3) near a +1 cell) converge to higher utility values, while states farther from positive rewards or closer to negative rewards (e.g. state (1,1)) settle at lower utilities. The plot confirms that after about 80 iterations, the utilities get close to their final values.

1.2. Method of Policy Iteration

1.2.a. Description of implemented solution

The same grid, transition model, and reward structure are used. The difference for Policy Iteration is the two alternating steps until policy stabilises:

- Policy Evaluation: Given a policy π, the utilities are computed by solving the linear system:

  $$U(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) \cdot U(s')$$

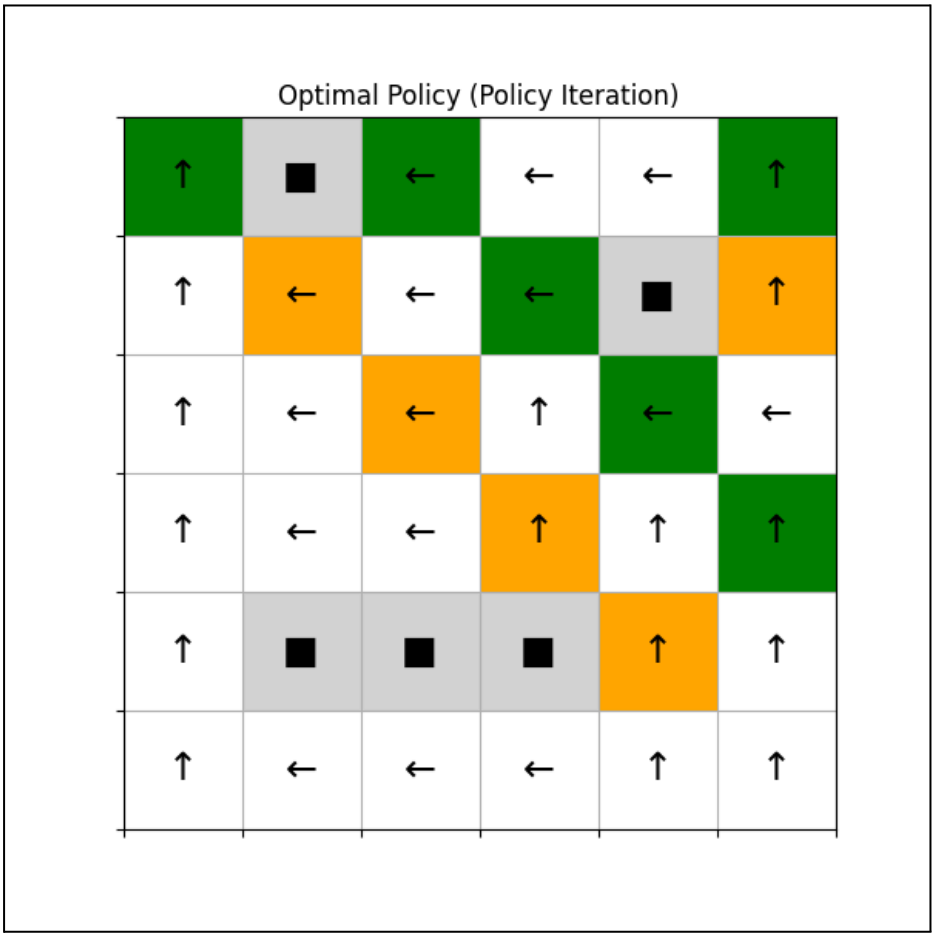
  I perform iterative updates (similar to value iteration but with a fixed policy) until convergences (change < 10^{-4}).
- Policy Improvement: For each state, the action that maximises the expected utility is selected.

  $$\pi_{i+1}(s) = \max_{a \in \{U,D,L,R\}} \sum_{s'} P(s'|s, a) \cdot U(s')$$

  If this action differs from the old policy for any state, the policy is updated and the process repeats.

The algorithm starts with a random policy (excluding walls). It also converges in far fewer outer loops than value iteration requires, this is because each policy evaluation itself uses many inner iterations. For this environment, policy iteration converged after only 3-4 iterations. Utilities of the same five states were recorded after each complete policy evaluation to produce the plot below.

1.2.b. Plot of optimal policy



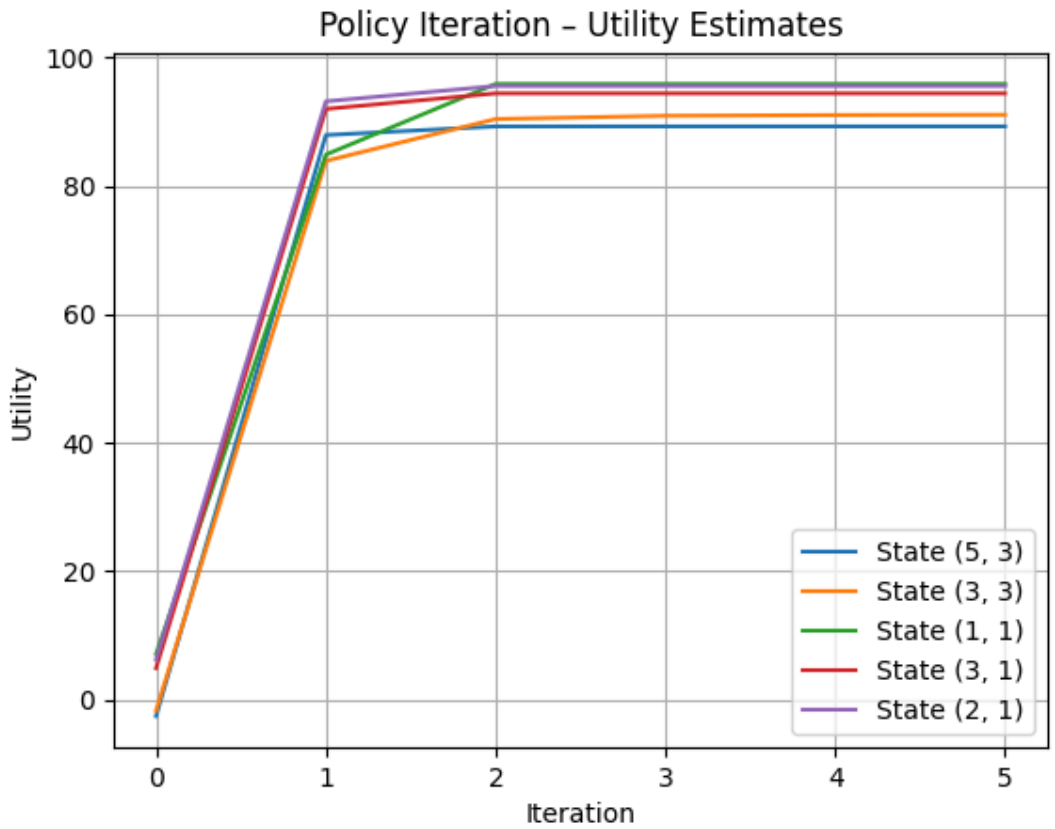
The optimal policy obtained using Policy Iteration is identical to the one obtained using Value Iteration, this tells us that both algorithms converge to the same optimal solution.

1.2.c. Utilities of all states from Policy Iteration

100	0	95.02	93.838	92.605	93.283
98.381	95.868	94.516	94.367	0	90.874
96.922	95.549	93.256	93.134	93.059	91.741
95.514	94.402	93.173	91.071	91.761	91.835
94.261	0	0	0	89.494	90.503
92.872	91.652	90.447	89.257	88.499	89.223

The utilities from value iteration and policy iteration are similar, differing only in the second decimal place (e.g. 95.01 vs. 95.02). This indicates that both algorithms have converged to the same optimal utility function.

1.2.d. Plot of utility estimates as a function of the number of iterations



The figure shows the utility estimates after each policy evaluation phase. Unlike the smooth, curved progression of value iteration, policy iteration shows only a few points. This behavior is because each evaluation computes the exact utilities for the current policy, and improvement then updates the policy, leading to rapid convergence. The final utilities are identical to those obtained in value iteration.

This plot also confirms that after 3 outer loops, the utilities have essentially reached their optimal values, demonstrating the efficiency of policy iteration.

2. Part 2 Questions

2.1. Design a more complicated maze environment of your own.

To find out the effect of state space size and environmental complexity on the convergence of value iteration and policy iteration, I designed a random maze generator.

The generator produces a grid of arbitrary dimensions, where each cell follows the maze variables in the assignment (Wall = None, White Square = -0.05, Green Square = +1, orange Square = -1). The generation procedure utilizes the following controllable variables:

- Grid size: Four different grids of size 10x10, 15x15, 20x20, and 25x25 were generated.
- Walls: Each cell has a fixed probability of 0.2 of being a wall
- Terminal rewards: A fixed no. of green and orange cells are placed randomly. In all experiments we used 5 green and 5 orange cells.

Note: The same probabilistic transition in Part 1 with intended direction having a 0.8 probability and right-angle directions each with probability 0.1 are used as well.

The following figures show the optimal policies of all mazes using value iteration and policy iteration.

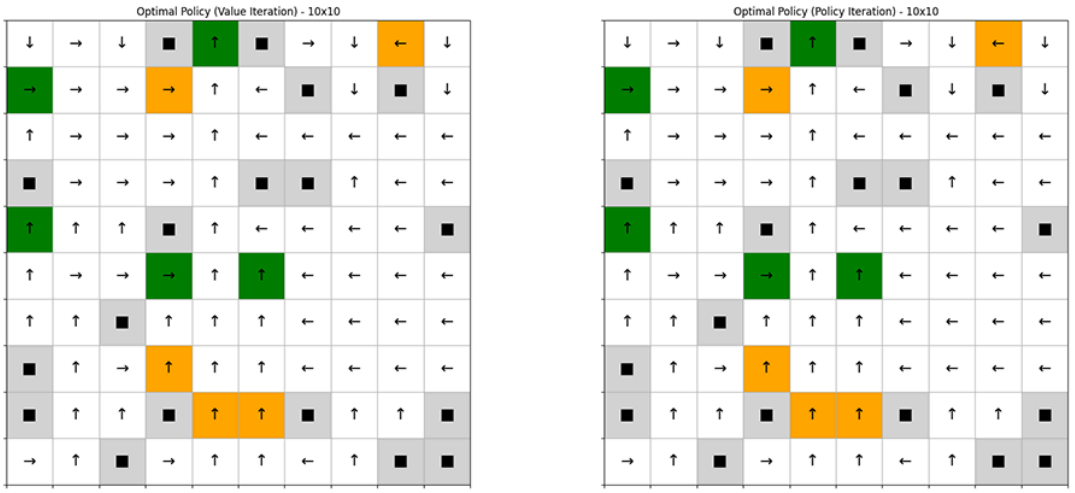


Figure 1: 10x10 Maze Optimal Policies for Value and Policy Iteration

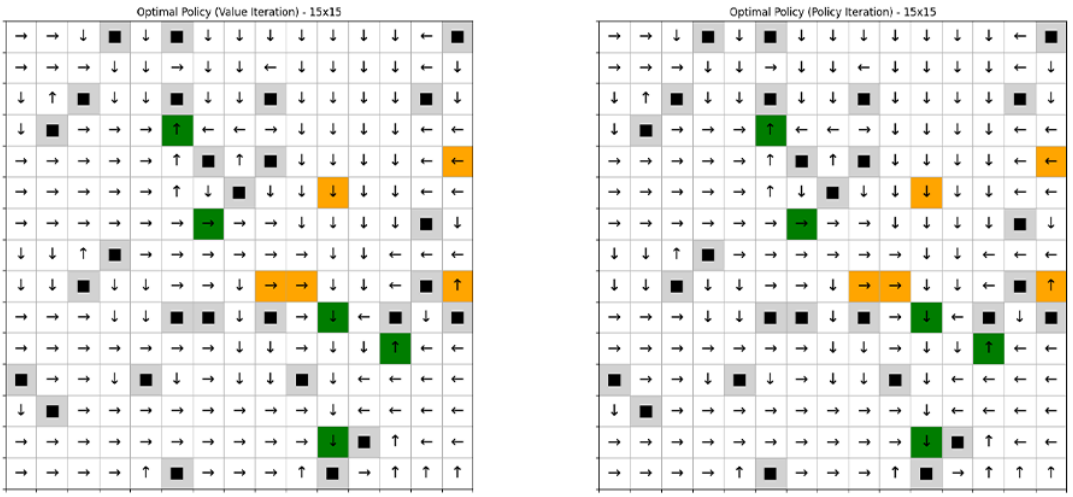


Figure 2: 15x15 Maze Optimal Policies for Value and Policy Iteration

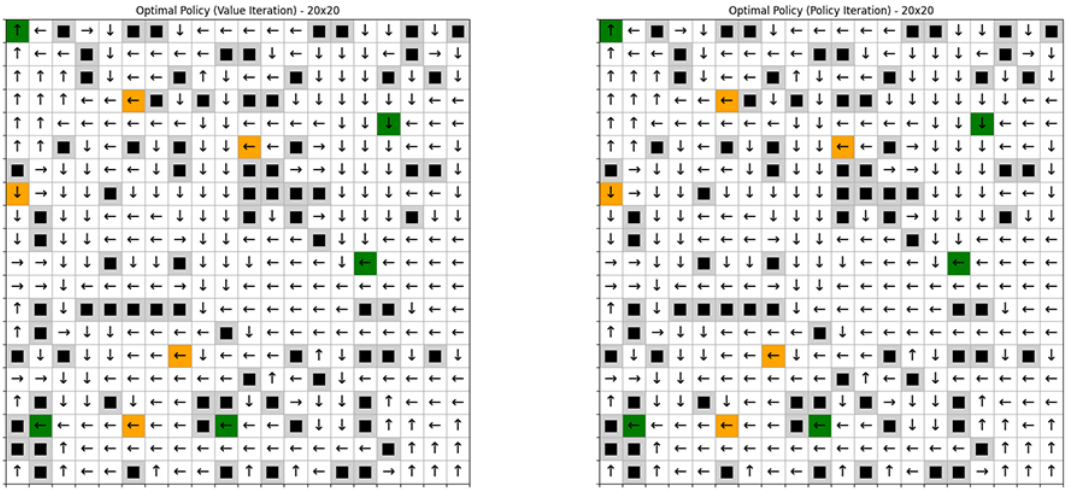


Figure 3: 20x20 Maze Optimal Policies for Value and Policy Iteration

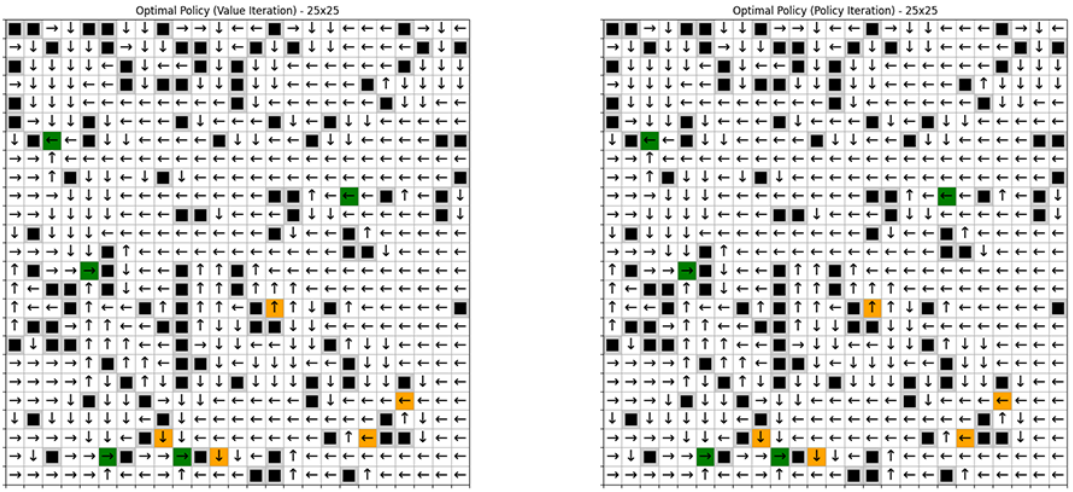


Figure 4: 25x25 Maze Optimal Policies for Value and Policy Iteration

2.2. How does the number of states and complexity of the environment affect convergence?

To observe how no. of states and complexity of the maze environment could affect convergence, I utilized four different grids with increasing scale to examine the effects. For each run I recorded:

- No. of iterations (or outer loops) required to reach a tolerance of 10^{-4} on the utility change
- Runtime
- Values of utility estimates for 5 randomly selected non-wall states
- Average utility over all non-wall states as a function of iteration

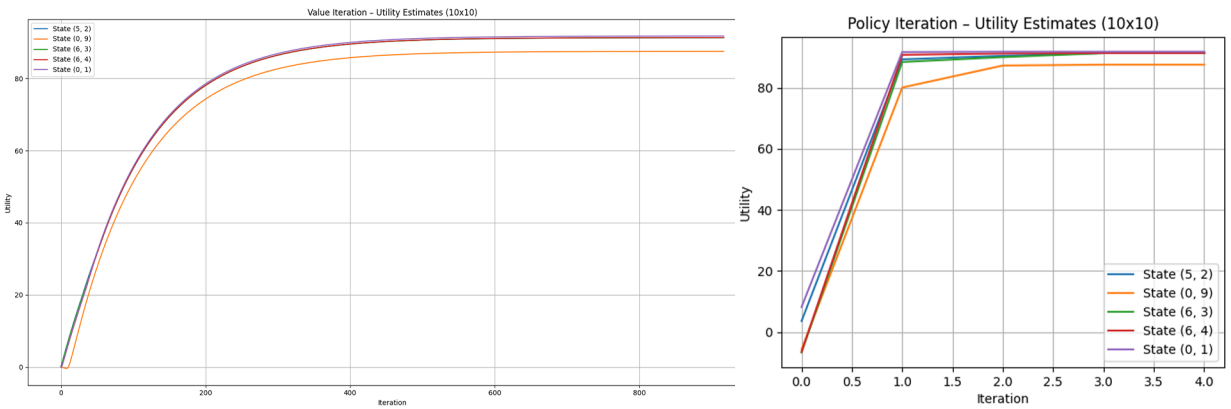


Figure 5: 10x10 Maze Utility Estimates over Iteration (Value and Policy Iteration)

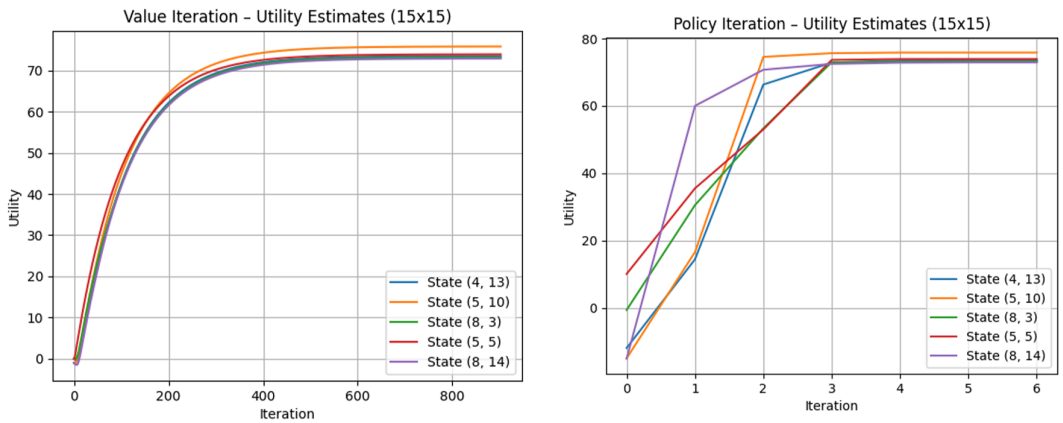


Figure 6: 15x15 Maze Utility Estimates over Iteration (Value and Policy Iteration)

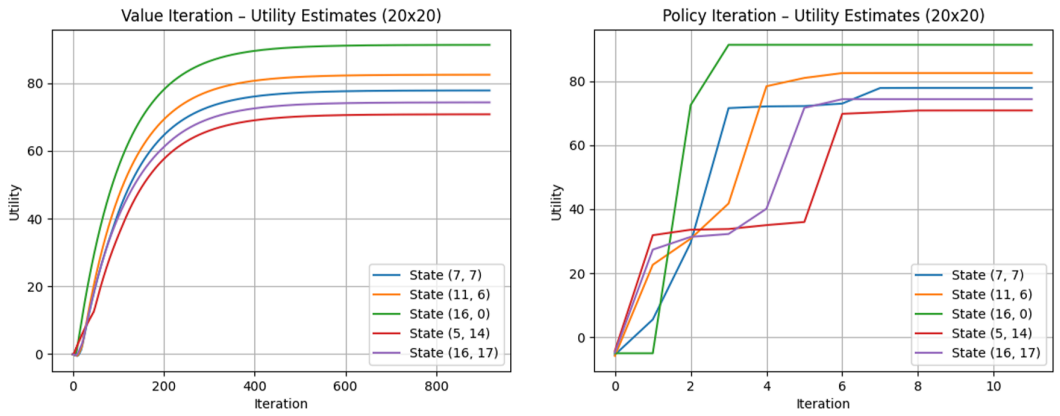


Figure 7: 20x20 Maze Utility Estimates over Iteration (Value and Policy Iteration)

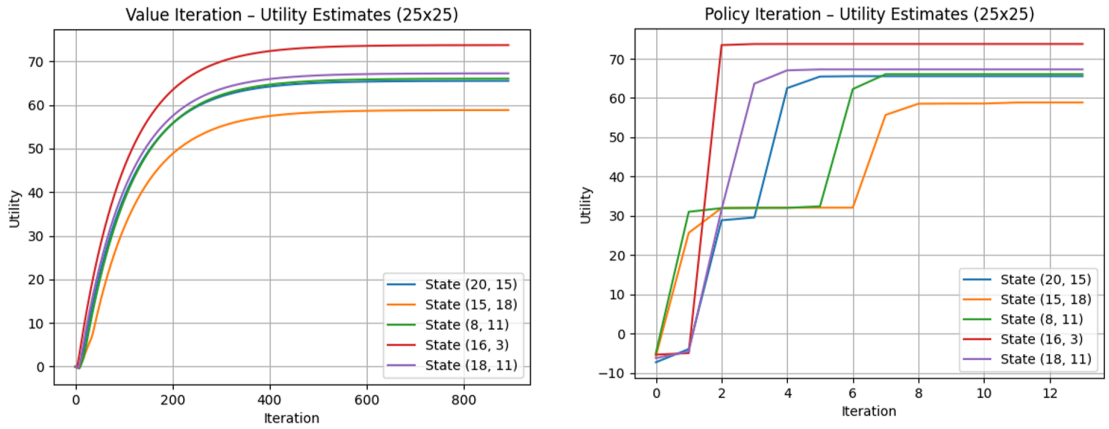


Figure 8: 25x25 Maze Utility Estimates over Iteration (Value and Policy Iteration)

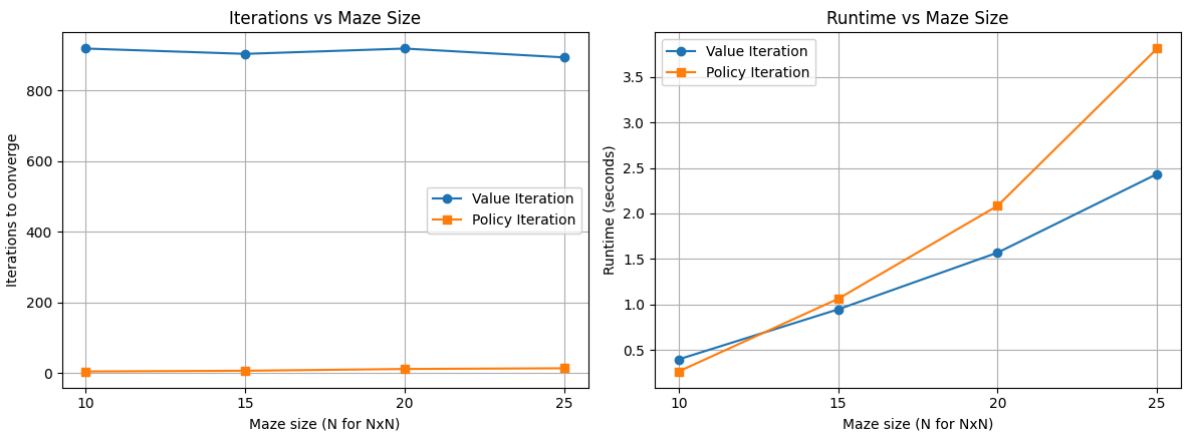


Figure 9: Comparison between the different Maze Sizes for Iterations and Runtime

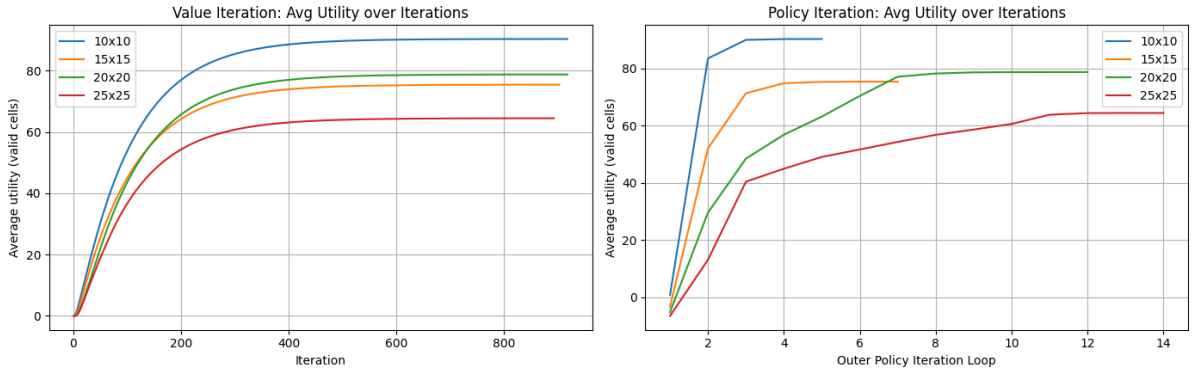


Figure 10: Comparison between Average Utility over Iterations (Value and Policy Iterations)

These were the observations made:

- <u>Value iteration:</u> (Figure 9) No. of iterations to converge is roughly the same no matter the grid size. However, the time taken to run all iterations grows with the grid size. This increase is expected because information about distant rewards must spread across the grid.

Grid size	**10x10**	**15x15**	**20x20**	**25x25**
No. of iterations	918	903	918	893
Runtime	0.40s	0.95s	1.57s	2.43s

- <u>Policy iteration:</u> (Figure 9) No. of outer loops required grows with the grid size. Consequently, the runtime of policy iteration becomes slower as the grid size increases.

Grid size	**10x10**	**15x15**	**20x20**	**25x25**
No. of outer loops	5	7	12	14
Runtime	0.26s	1.06s	2.08s	3.81s

- <u>Average utility:</u> (Figure 10) Larger grids show a slower initial increase because utilities of many cells start at zero and need more updates to reflect distant rewards.

- <u>Utility of individual states:</u> (Figures 6, 7, 8) The utility of the individual states closer to positive rewards stabilise earlier, while those far from any terminal reward take longer to converge. The transition model also causes minor fluctuations before final convergence.

2.3. How complex can you make the environment and still be able to learn the right policy?

To figure out how complex the environment can get, I extended the random maze generator (while keeping the same transition model and reward structure) to produce grids up from 30x30 to 100x100 with the same 0.2 wall probability but with 20 green cells and 20 orange cells. The exact same value and policy iteration code from Part 2 is used, with discount factor at 10^{-4}.

All the tested mazes converged successfully. Although the absolute utility values naturally differ between maze sizes, the value and policy iteration algorithms still produce identical optimal solutions within each individual maze. The no. of iterations and runtime are recorded in the following table:

Value Iteration

Grid size	**30x30**	**40x40**	**50x50**	**100x100**
No. of iterations	918	891	904	890
Runtime	3.57s	5.77s	9.10s	36.76s

Policy Iteration

Grid size	**30x30**	**40x40**	**50x50**	**100x100**
No. of outer loops	17	23	16	18
Runtime	5.04s	10.10s	11.30s	65.42s

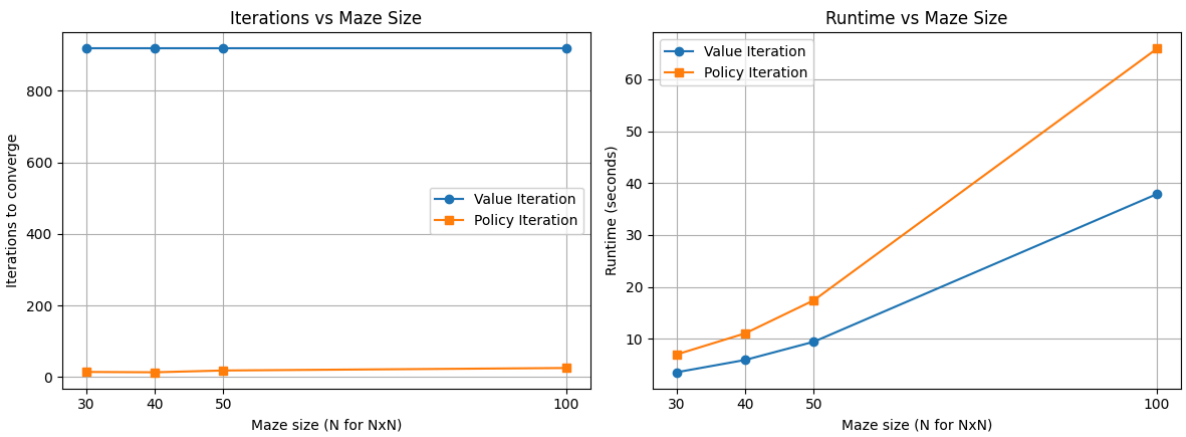


Figure 11: Comparison between the Bigger Mazes for Iterations and Runtime

As seen, when I tried an extreme 100x100 grid, both algorithms still converged reliably, but policy iteration exceeded 60 seconds on my computer. This demonstrates the expected scaling: each iteration must visit every state, so runtime grows with no. of non-wall states.

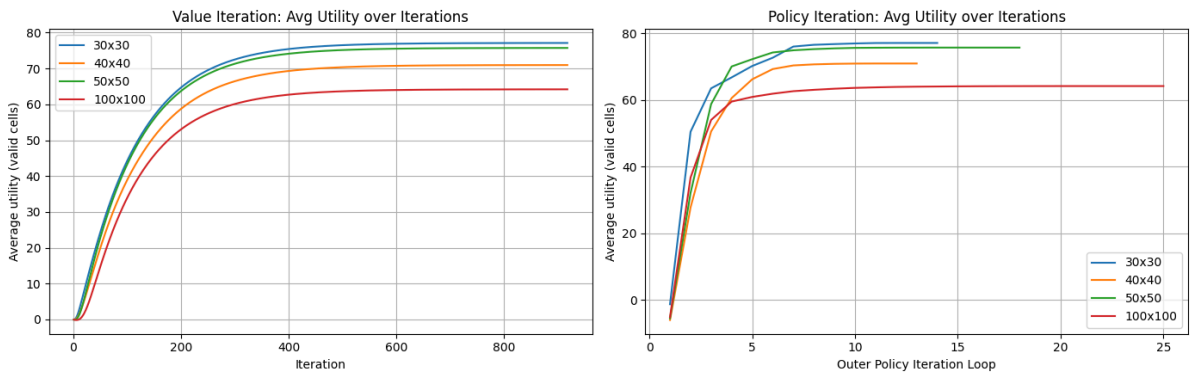


Figure 12: Comparison between Bigger Mazes on Average Utility over Iterations (Value and Policy Iterations)

The average-utility curves in Figure 12 rise smoothly and flattens at stable values in every grid, confirming convergence. The slight difference in final average utility across the different grid sizes is expected as larger grids simply contain more cells whose utilities are lower because of the fixed no. terminal states, becoming more and more sparsely distributed as we increase the grid size.

<u>Conclusion</u>

Theoretically, there should be no upper bound on the environmental complexity. As long as the state space is finite, both value iteration and policy iteration are guaranteed to converge. The only limitation are computational resources, as on my computer (standard hardware), 100x100 grids are solvable but already take a longer runtime to compute. In all test cases tested, the algorithms reliably learned the right optimal policy.